

2143 Object-Oriented Programming

Exam 03 — Spring 2026 | 80 minutes | 100 points

Instructions:

- Record all answers on the **Answer Sheet**. Nothing written on this booklet will be graded.
- For Part I circle the single best answer on the MC grid.
- For Part III write clearly — partial credit is awarded.
- Assume using namespace `std`; and necessary headers included unless stated otherwise.
- No electronic devices.

1 Multiple Choice (2 pts each — circle the single best answer on the Answer Sheet)

- (2 pts) Runtime polymorphism in C++ requires which **two** conditions to both be true?
 - A. The function is declared `static` in the base class.
 - B. The function is declared `virtual` in the base class **and** the call is made through a base class pointer or reference.
 - C. The derived class uses the `override` keyword.
 - D. The base class must be abstract.
- (2 pts) What is **object slicing**?
 - A. Removing unused methods from a class at compile time.
 - B. When a derived object is assigned to a base object by value, losing derived-specific data and behavior.
 - C. A compiler optimization that trims the vtable.
 - D. Splitting a large class into multiple smaller ones.
- (2 pts) Given the code below, which version of `greet()` executes?

```
Base* p = new Derived();    // greet() is virtual and overridden in Derived
p->greet();
```

 - A. `Base::greet()`
 - B. `Derived::greet()`
 - C. Both run in sequence, base first.
 - D. Undefined behavior — must cast to `Derived*` first.
- (2 pts) Which of the following call sites uses **static** (compile-time) dispatch?
 - A. `Base* p = new Derived(); p->foo();`
 - B. `void f(Base& b) { b.foo(); }`
 - C. `Derived d; d.foo();`
 - D. `Base& r = d; r.foo();`
- (2 pts) `delete a;` is called where `Animal* a = new Dog();`. The destructor `~Animal()` is **not** virtual. What happens?

- A. Both destructors run in the correct order.
 - B. Only `~Dog()` runs.
 - C. Only `~Animal()` runs — derived resources may leak.
 - D. A compile-time error occurs.
6. (2 pts) What makes a C++ class **abstract**?
- A. Declaring it with the **abstract** keyword.
 - B. Having at least one pure virtual function (`= 0`).
 - C. Making all data members **private**.
 - D. Inheriting from another class.
7. (2 pts) Which is a correctly declared **pure virtual** function?
- A. `virtual void draw() const;`
 - B. `void draw() const override;`
 - C. `virtual void draw() const = 0;`
 - D. `static virtual void draw() = 0;`
8. (2 pts) `Shape` declares `virtual double area() const = 0;`. `Circle` inherits `Shape` but does **not** override `area()`. What is true?
- A. `Circle` inherits a default `area()` returning 0.
 - B. `Circle` is also abstract and cannot be instantiated.
 - C. The program crashes at runtime when `area()` is called on a `Circle`.
 - D. `Circle::area()` returns garbage.
9. (2 pts) Given abstract `Animal` (pure virtual `speak()`) and `Dog` (overrides `speak()`), which line is **legal**?
- A. `Animal a;`
 - B. `Animal* p = new Animal();`
 - C. `Animal* p = new Dog();`
 - D. `Animal a = Dog();`
10. (2 pts) A class intended to serve as a base class should **always** include:
- A. A static factory method.
 - B. A **protected** default constructor.
 - C. A virtual destructor.
 - D. At least two pure virtual functions.
11. (2 pts) Which statement about **static member functions** is true?
- A. They have access to the **this** pointer.
 - B. They can only be called through an object instance.
 - C. They cannot directly access non-static data members.
 - D. Every class must define at least one.
12. (2 pts) A static data member is declared inside the class body. Where must it also be **defined**?

- A. Directly after the declaration, inside the class body.
 - B. In every `.cpp` file that uses the class.
 - C. Outside the class body exactly once — e.g., `int Widget::count = 0;`
 - D. In the class constructor.
13. (2 pts) `Widget::getCount()` is called without creating any `Widget` object. This is valid because `getCount()` is declared:
- A. `friend` (B) `virtual` (C) `static` (D) `const`
14. (2 pts) Which relationship is best modeled with **composition** rather than inheritance?
- A. A `Square` and a `Shape`
 - B. A `Dog` and an `Animal`
 - C. A `Car` and an `Engine`
 - D. A `SavingsAccount` and an `Account`
15. (2 pts) The design principle “favor composition over inheritance” is recommended because composition:
- A. Is always faster at runtime.
 - B. Is more flexible — parts can be swapped and class coupling is reduced.
 - C. Automatically prevents the diamond problem.
 - D. Always produces shorter code.
16. (2 pts) Which phrase describes an **inheritance** (“is-a”) relationship?
- A. A `House` has a `Room`.
 - B. A `Computer` has a `CPU`.
 - C. A `SportsCar` is a `Car`.
 - D. A `Library` contains `Book` objects.
17. (2 pts) A `Library` object holds a collection of `Book` objects. This design is an example of:
- A. Inheritance (B) Composition (C) Polymorphism (D) Abstraction
18. (2 pts) The `protected` access specifier allows a member to be accessed from:
- A. Only the class itself.
 - B. The class and its derived classes, but not external code.
 - C. All code, including external functions.
 - D. Only `friend` classes and functions.
19. (2 pts) Inside a derived class, which members of the **base** class can be accessed directly?
- A. Only `public` members.
 - B. `public`, `protected`, and `private` members.
 - C. `public` and `protected` members only.
 - D. Only `protected` members.
20. (2 pts) A virtual destructor is **required** when:

- A. Any class contains dynamic memory.
 - B. A class has more than three data members.
 - C. An object may be **deleted** through a base class pointer.
 - D. A class declares any **virtual** function.
21. (2 pts) In the hierarchy $A \rightarrow B \rightarrow C$ (A is root), a C object is created then goes out of scope. What is the correct order of calls?
- A. C ctor, B ctor, A ctor, C dtor, B dtor, A dtor
 - B. A ctor, B ctor, C ctor, A dtor, B dtor, C dtor
 - C. A ctor, B ctor, C ctor, C dtor, B dtor, A dtor
 - D. C ctor, B ctor, A ctor, A dtor, B dtor, C dtor
22. (2 pts) The primary purpose of the **override** keyword (C++11) is to:
- A. Force the compiler to inline the function.
 - B. Make the function virtual automatically.
 - C. Have the compiler verify the function actually overrides a base virtual — catching typos and signature mismatches.
 - D. Prevent the function from being overridden in deeper subclasses.
23. (2 pts) The **diamond problem** in C++ is resolved using:
- A. The **abstract** keyword (B) The **final** keyword
 - B. **virtual** inheritance (D) Multiple **override** declarations
24. (2 pts) What happens to an exception that is **never caught** anywhere in the program?
- A. It is silently ignored.
 - B. It becomes a compiler warning on the next build.
 - C. `std::terminate()` is called, typically ending the program immediately.
 - D. It is returned as the exit code of `main`.
25. (2 pts) Which smart pointer enforces **exclusive** (sole) ownership and **cannot** be copied?
- A. `std::shared_ptr` (B) `std::weak_ptr`
 - B. `std::unique_ptr` (D) `std::auto_ptr`
-

2 Matching (1 pt each — write the **letter** on the Answer Sheet)

Terms (left)

Descriptions (right)

- | | |
|--------------------------|--|
| 1. <code>virtual</code> | |
| 2. <code>override</code> | |
| 3. <code>final</code> | |
| 4. <code>= 0</code> | |
| 5. <code>friend</code> | |
| 6. <code>throw</code> | A. Compiler-enforced check that a method actually overrides a base class virtual. |
| 7. <code>catch</code> | B. Signals that an exceptional situation has occurred; propagates up the call stack. |
| 8. <code>template</code> | C. Begins a generic, type-parameterized function or class definition. |
| 9. <code>RAII</code> | D. Makes a virtual function pure — the containing class becomes abstract. |
| 10. Object Slicing | E. Prevents a virtual function from being further overridden, or a class from being inherited. |
| | F. Grants an outside function or class direct access to <code>private</code> members. |
| | G. Handles a thrown exception; matched by type. |
| | H. Ties a resource's lifetime to a stack object's lifetime via constructors and destructors. |
| | I. Marks a method as participating in runtime (dynamic) dispatch. |
| | J. Loss of derived-class data when a derived object is copied to a base variable by value. |
-

3 Code Tracing & Writing (write all answers on the Answer Sheet)

1. (10 pts) Trace the Output.

Study the program below. On the Answer Sheet write the **exact output**, one entry per numbered line. Annotate *why* beside any line you are unsure of — annotated wrong answers may earn partial credit.

```

1 class Base {
2 public:
3     virtual void greet() { cout << "Hello from Base\n"; }
4     virtual ~Base()      { cout << "~Base\n"; }
5 };
6 class Derived : public Base {
7 public:
8     void greet() override { cout << "Hello from Derived\n"; }
9     ~Derived()            { cout << "~Derived\n"; }
10 };
11
12 void test1(Base b) { b.greet(); }    // by value
13 void test2(Base& b) { b.greet(); }   // by reference
14
15 int main() {
16     Derived d;
17     test1(d);
18     test2(d);
19     Base* p = new Derived();
20     p->greet();
21     delete p;
22 }

```

2. (10 pts) Find the Bug.

The program below compiles but has a serious defect. On the Answer Sheet:

(A) Identify and explain the bug in 1–2 sentences. (4 pts)

(B) Write the minimum one-line fix. (3 pts)

(C) After your fix, write the exact output when `main()` runs. (3 pts)

```

1 class Animal {
2 public:
3     Animal() { cout << "Animal ctor\n"; }
4     ~Animal() { cout << "Animal dtor\n"; }    // <-- examine this
5 };
6 class Dog : public Animal {
7     int* tricks;
8 public:
9     Dog() { tricks = new int[50]; cout << "Dog ctor\n"; }
10    ~Dog() { delete[] tricks; cout << "Dog dtor\n"; }
11 };
12 int main() {
13     Animal* a = new Dog();
14     delete a;
15 }

```

3. (10 pts) **Design & Write Code.**

A game has **Player** objects. Each **Player** *has-a* **Weapon**. The game engine needs to know the total number of active **Player** objects at any time without iterating a list.

On the Answer Sheet:

Part A — Weapon Class (3 pts).

Write a **Weapon** class with `string name` and `attack()` that prints "*name* attacks!".

Part B — Player Class (7 pts).

Write a **Player** class using **composition** to hold a **Weapon**, with a **static** `count` member (incremented in ctor, decremented in dtor) and a static `getCount()` method. Also write the one line required **outside** the class to define `count`.

4. (10 pts) **Templates & Abstraction.**

On the Answer Sheet:

Part A — Template Trace (4 pts).

Write the **exact output** of the program below.

```
1 template <typename T>
2 T bigger(T a, T b) { return (a > b) ? a : b; }
3
4 int main() {
5     cout << bigger(3, 7) << "\n";
6     cout << bigger(3.14, 2.71) << "\n";
7     cout << bigger(string("apple"), string("banana")) << "\n";
8 }
```

Part B — Abstract Class Hierarchy (6 pts).

On the Answer Sheet write an **abstract** base class **Shape** with: a pure virtual `double area()` `const`, a virtual destructor, and a non-virtual `describe()` that prints "Area: <area()>". Then write a concrete **Rectangle** that stores `width` and `height` and overrides `area()`.